

Scaling Scaling Laws with Board Games

Scaling scaling laws with board games

For individual researchers such as myself, it can be tough to do research. I don't have infrastructure, compute, and am severely bottlenecked in these aspects which make it difficult to do full research (exmaple, when I tried replicating and extending Anthropic's Constitutional AI on a limited 30GB disk quota constraint on my university's HPC). While I appreciated the fact that I even had an HPC in the first place, it was no way to do proper research. But, this paper covers the fact that oftentimes, such things are not even needed! Let's explore what the author means!

The blessing of scaling

We have observed that the scaling laws are a real phenomena with LLMs and it is relatively accurate to predict things on the basis of where on the curves we pick to do analysis on. So, if that were to be true, we could do experiments using smaller, cheaper models, extract key results and scale up the results such that we'd know that they'd hold up on the basis of the scaling laws. This is actually an awesome argument, and tells us an important fact: **you don't need to wait to have amazing resources to do novel and important experimentations, as long as you can show that the results hold at some point in the curve. The results could be replicated later down the line, and, in fact, it is predictable that they would!**

While this is true, it is also not necessarily the case because scaling does seem to do something qualitatively different too. Example, I saw a self-reinforcing attention head that was not present in Gemma2B versus Gemma12B-IT, so it is important to not take this idea too far. It seems, some ideas can only be pursued precisely when you have access to such resources. But, that doesn't mean **every experiment that is important in AI right now** falls into that category!

In this paper, the authors pick the task of playing the board game Hex and using AlphaZero to play it.

This paper is interesting because, it looks at measuring and developing relationships in all of us on the basis of multi-agent RL settings, which are very different to single-agent settings. In zero sum games, the metric used to measure performance of the system depends on the task and the other agent(s) too. Example, for chess, ELO scores are calculated, not only for the current agent but also all the other agent(s) it competes against: the likely winner is the one who has the bigger score, so the difference between the two values is used.

Methods and Results

The authors developed fast versions of AlphaZero and trained many different models using it (you need to experiment doing precisely this. PropperShopper would be a good first environment to work on!). The authors used machines with GPU rented from vast.ai. Using Lambda would be a good, cheap alternative too.

When AlphaGo agents are trained to play Hex perfectly, an interesting relationship arises. You can estimate how much compute you need, down to the level of how much for training versus testing, depending on the Hex board size and the level of expertise you want the agents to exhibit. That level of precision is an interesting result, no doubt.

While the authors drew heavily inspirations from open source implementations of AlphaZero, they successfully scaled it down, as their experiment was a narrowed and less complex task than superhuman play at Go. They also wrote the code to work completely on the GPU, whereas there are some components of AlphaZero (the environment and the MCTS bits) that were CPU based.

MCTS is actually CPU bound. MCTS is a sequential tree search — each simulation depends on the result of the previous one to decide where to explore next. The UCB selection formula at each node:

$$UCT = Q(s, a) + c \cdot P(s, a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

requires knowing the current visit counts before selecting the next node. You can't parallelize this the way you parallelize matrix multiplications — each step has a data dependency on the previous step.